

Frontend Engineering Guide: Role, Technologies & Libraries

Comprehensive guide to UI architecture, the 2-canvas sandwich model, core libraries, state synchronization, and client-side coordinate transformation for the React.js PDF Editor.

1. The Core Role of the Frontend

In an in-place PDF editor, the frontend handles **75% of the user experience and interactive logic**. Because PDF documents use fixed vector coordinates rather than reflowable DOM elements, the frontend implements an interactive rendering and overlay pipeline.

1. Document Rendering & Viewport Control

- Converts uploaded PDF binary streams into crisp visual pages rendered onto HTML5 `<canvas>` elements.
- Controls dynamic zoom scaling (50% to 200%), page navigation, sidebars, and viewport scrolling.

2. Interactive Canvas Overlay (Layer 3)

- Mounts a transparent vector drawing board directly over each page.
- Allows users to click, drop draggable textboxes, resize elements, and draw whiteout redaction rectangles.

3. UI Toolbars & State Synchronization

- Controls active tools (Pointer, Text, Erase, Draw, AI Copilot, Download).
- Maintains multi-page edit history, font choices, colors, and undo/redo stacks.

4. Coordinate Transformation & Stamping

- Translates screen CSS pixel coordinates into standard 72 DPI PDF points.
- Bakes all text and whiteout edits directly into the original PDF binary file for instant download.

2. Frontend Technologies & Libraries Matrix

Library / Package	Type	Specific Role & Responsibility	Why It Is Essential
React.js (with Vite)	UI Framework	Manages component hierarchy, active tool states, zoom scale, file upload handlers, and responsive toolbar controls.	Avoids DOM spaghetti; provides declarative state management.
pdfjs-dist (Mozilla)	PDF Parsing & Rendering	The "Eyes" of the app. Reads binary PDF data and executes 2D canvas draw instructions to render document pages visually.	Enables in-browser document display without external plugins.
fabric (Fabric.js)	Interactive Canvas Layer	The "Interaction Engine". Mounts on top of each page to provide movable textboxes (IText), whiteout boxes (Rect), and vector drawing.	Provides built-in drag, resize, rotation, and click hit-testing.
pdf-lib	PDF Mutation & Export	The "Hands" of the app. Takes original PDF bytes, burns whiteout rectangles, stamps new text at exact coordinates, and exports the final PDF.	Pure client-side PDF modification; zero server load for export.
lucide-react	Icon System	Provides crisp SVG vector icons for tools: Type, Eraser, Pen, Sparkles, Download, ZoomIn.	Polished, lightweight toolbar user interface.
tailwindcss	Styling Framework	Utility classes for styling toolbars, floating AI popovers, modals, dropzones, and responsive layout shells.	Fast, consistent, modern UI design without writing raw CSS.

3. Clear Comparison: `pdfjs-dist` vs. `Fabric.js` vs. `pdf-lib`

Feature	pdfjs-dist (Viewer)	Fabric.js (Overlay)	pdf-lib (Exporter)
Primary Responsibility	Reads and renders original PDF to screen.	Captures user clicks, text typing, and drag/drop edits.	Modifies binary PDF bytes and saves the final file.
Canvas Usage	Paints visual pixels onto bottom <canvas>.	Operates on transparent top <canvas>.	No canvas; operates purely in memory on byte buffers.
Can it edit PDFs?	No (Read-only viewer engine).	No (Only manages browser canvas vectors).	Yes (Writes vector & text streams to PDF).
User Interaction	Static background display.	Fully interactive (drag, resize, focus).	Non-interactive (runs on download/export).

4. The 2-Canvas Page Component Architecture

Each PDF page in the document viewer is rendered as a standalone React component containing two perfectly synchronized canvas elements stacked using CSS:

```
<!-- HTML / DOM Hierarchy for each PDF Page -->
<div className="relative shadow-md border bg-white" style={{ width: 800, height: 1100 }}>

  <!-- LAYER 1: Background Render (pdfjs-dist) -->
  <canvas ref={pdfCanvasRef} className="block" />

  <!-- LAYER 2: Interactive Editing Overlay (Fabric.js) -->
  <div className="absolute top-0 left-0">
    <canvas ref={fabricCanvasRef} />
  </div>

</div>
```

React Lifecycle & Canvas Protection Rule

Because React re-renders can destroy and recreate DOM elements, the Fabric.js canvas instance is stored inside a `useRef(null)` hook. This ensures that when toolbar state changes (like switching tools or adjusting font color), the canvas object tree and active user textboxes are **never wiped or re-initialized**.

5. Step-by-Step Frontend Implementation Plan

Phase 1: Project Setup & Viewer (Days 1–3)

- Scaffold React + Vite project with Tailwind CSS.
- Install `pdfjs-dist`, `fabric`, `pdf-lib`, `lucide-react`.
- Configure PDF.js Web Worker path in `main.jsx`.
- Build drag-and-drop file uploader accepting `.pdf`.
- Implement `PDFViewer.jsx` to render all pages sequentially with zoom controls (50% to 200%).

Phase 2: Editing Overlay & Tools (Days 4–7)

- Mount Fabric.js canvas transparently over each page.
- Build `TopToolbar.jsx` with active tool buttons: Select, Add Text, Whiteout (Erase), Draw.
- Add text styling controls: font size, color, font family.
- Implement Undo / Redo history state stack.

Phase 3: Export & Coordinate Math (Days 8–10)

- Implement `coordinates.js` to transform DOM screen pixels to 72 DPI PDF points.
- Build `exporter.js` using `pdf-lib` to draw white rectangles and stamp replacement text.
- Validate that exported PDFs match original layouts at all zoom levels.

Phase 4: AI Copilot Modal & Polish (Days 11–14)

- Build floating AI Rewrite modal triggered on text selection.
- Connect frontend to backend `POST /api/ai/rewrite`.
- Automate whiteout box creation and replacement text insertion upon receiving AI response.
- Final UI polish, responsive toolbar, and defense demo prep.

6. Frontend Documentation & Learning References

Technology	Documentation Link	What to Read First
PDF.js (Mozilla)	mozilla.github.io/pdf.js/	Read <code>getDocument()</code> , <code>getViewport()</code> , and <code>page.render()</code> examples.
Fabric.js	fabricjs.com/demos/	Read interactive object tutorials: <code>IText</code> , <code>Rect</code> , and canvas event listeners.
pdf-lib	pdf-lib.js.org/	Read "Modify Document" guide: <code>page.drawText()</code> and <code>page.drawRectangle()</code> .
Lucide Icons	lucide.dev/icons/	Search icons for your top editor toolbar.